



VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

University of Science
Faculty of Information Technology

LABORATORY REPORT

Decision Tree Modeling and Improvement

*Student Performance Prediction – Pass/Fail Classification on Kaggle
Dataset*

Coursework	Introduction to Artificial Intelligence – Lab 2
Report status	Final version
Prepared on	September 2026
Repository	github.com/AdrianParry-17/AI-Lab-Decision-Tree

GROUP 6

Class / section 24C03

Nguyen Minh Duc	24127345 – Visualize, Analysis
Van Phu Duc	24127346 – Video
Huynh Minh Hung	24127385 – Train, Improvement
Hy Hue Hung	24127388 – Dataset, Report

Contents

1. Group Introduction	1
2. Introduction	1
2.1. Decision Trees	1
2.2. Project Objective	2
3. Dataset Description	2
3.1. Source	2
3.2. Samples, Features and Target	2
3.3. Preprocessing	3
3.4. Why This Dataset Fits Decision Trees	3
4. Baseline Model	4
4.1. Model Description	4
4.2. Training and Testing Procedure	4
4.3. Resulting Tree	4
4.4. Accuracy and Error Rate	5
5. Analysis of the Resulting Tree	6
5.1. Overall Structure	6
5.2. Important Decision Rules	6
5.3. Feature Usage and Impurity Analysis	7
5.4. Strengths and Weaknesses	7
6. Improvement Methods	8
6.1. Improvement 1 – Depth Limiting (Gini, max_depth = 4) – Pre-pruning	8
6.2. Improvement 2 – Criterion Change (Gini to Entropy, depth = 4)	10
7. Comparison of Results	11
8. Conclusion	12
9. References	13
10. Appendices	14
10.1. Appendix A – Baseline Tree (detailed excerpt)	14
10.2. Appendix B – Depth-limited Gini Tree	14
10.3. Appendix C – Depth-limited Entropy Tree	14
10.4. Appendix D – Reproduction	14

1. Group Introduction

The contributions below reflect the group's final task allocation as required by Lab 2 - Decision Tree.pdf:4.

No.	Full Name	Student ID	Contribution
1	Nguyen Minh Duc	24127345	Visualize, Analysis
2	Van Phu Duc	24127346	Video presentation
3	Huynh Minh Hung	24127385	Train, Improvement
4	Hy Hue Hung	24127388	Dataset, Report

Table 1. Group members and contributions

Field	Value
Group name	GROUP 6
Class / section	24C03
Project repository	https://github.com/AdrianParry-17/AI-Lab-Decision-Tree
Report status	Final version

Table 2. Group information

2. Introduction

2.1. Decision Trees

A decision tree is a non-parametric supervised learning model that recursively partitions the data by choosing the attribute and threshold that best reduces impurity. Two common impurity measures for classification are Gini impurity and Entropy:

$$\text{Gini} = 1 - \sum_i p_i^2$$

$$\text{Entropy} = - \sum_i p_i \log_2 p_i$$

At each node the algorithm tests all possible splits and selects the one that minimizes weighted impurity. Splitting stops when a node becomes pure, no useful split remains, or a stopping criterion such as maximum depth is reached. Each leaf predicts the majority class of its training samples, so every root-to-leaf path can be read as an intuitive if-then rule. The custom implementation in this project handles categorical strings directly via value-based splits and numerical features via threshold splits.

Decision trees are especially attractive because they are interpretable, handle mixed-type features without scaling, and capture non-linear, conditional relationships.



Figure 1. Decision-tree pipeline: Input -> Split -> Leaf.

2.2. Project Objective

The objectives of this lab are to understand how a decision tree works, apply it to a real dataset, evaluate its performance with appropriate metrics, analyze the resulting tree, propose and test two to three improvements, and practice technical writing and teamwork.

3. Dataset Description

3.1. Source

The dataset used in this project is the public **Student Performance Prediction** dataset from Kaggle: <https://www.kaggle.com/datasets/amrmaree/student-performance-prediction>. A local copy is stored as `data/student_data.csv` for reproducibility. The task is binary classification: predicting whether a student will Pass or Fail.

3.2. Samples, Features and Target

Samples

Statistic	Count
Total rows	708
Distinct records	500
Duplicate rows (oversampled)	208
Training set (75%)	531 rows (375 distinct)
Testing set (25%)	177 rows (125 distinct)
Missing values	0

Table 3. Sample counts. Stratified split, seed 42, duplicates grouped to avoid leakage.

The overall dataset is perfectly balanced, with equal numbers of Fail and Pass samples. The training and testing partitions are nearly balanced, and duplicate-group verification shows zero overlap.

Partition	Fail	Pass	Total
Overall	354 (50.0%)	354 (50.0%)	708
Training	266 (50.1%)	265 (49.9%)	531
Testing	88 (49.7%)	89 (50.3%)	177

Table 4. Class distribution.

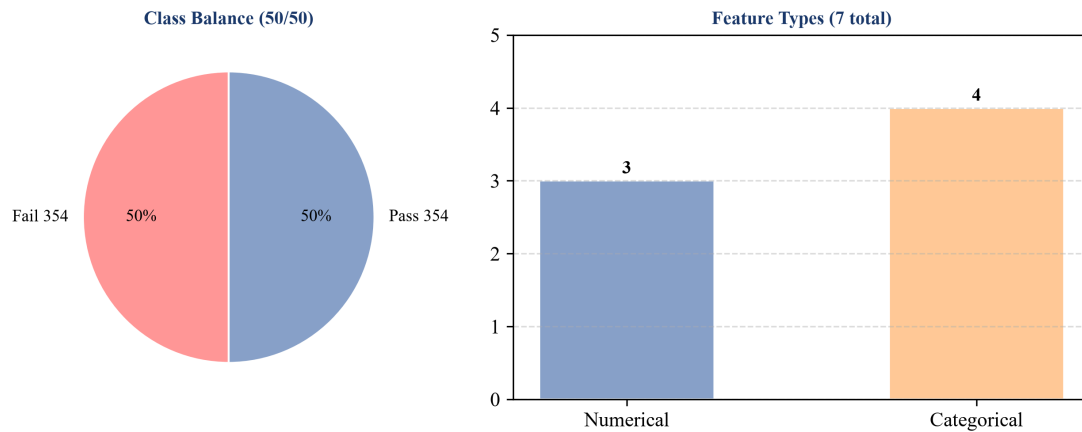


Figure 2. Dataset overview: balanced classes and feature types.

Features – 10 raw columns, 7 usable after excluding `Student_ID` and `Final_Exam_Score` (leakage):

Column	Type	Description
Gender	Categorical	Male, Female
Study_Hours_per_Week	Numerical	10–39, mean Pass 28.8, Fail 23.5
Attendance_Rate	Numerical	50.1–99.9, mean Pass 83.6, Fail 72.6
Past_Exam_Scores	Numerical	50–100, mean Pass 84.2, Fail 71.6
Parental_Education_Level	Categorical	High School, Bachelors, Masters, PhD
Internet_Access_at_Home	Categorical	Yes / No
Extracurricular_Activities	Categorical	Yes / No

Table 5. The 7 features used for modeling (3 numerical, 4 categorical).

Target variable: `Pass_Fail` with two classes, Pass and Fail.

3.3. Preprocessing

1. Validate header and required columns.
2. Remove `Student_ID` and `Final_Exam_Score`.
3. Convert numerical columns to int/float and keep categorical strings via `AttributeValueSplit` (no one-hot needed).
4. Check for missing values.
5. Stratified 75/25 split with seed 42, duplicates grouped.
6. No scaling needed.

3.4. Why This Dataset Fits Decision Trees

This dataset is well suited for decision trees. It is a classification problem where interpretability matters. It contains mixed numerical and categorical features with natural thresholds, which the custom tree handles directly. Study time and attendance interact conditionally with past scores, and trees model such non-linear interactions naturally. The size is moderate (708 rows, 500 distinct), large enough to learn but small enough to visualize and demonstrate pre-pruning (171 nodes depth 10 -> 35 nodes depth 4).

4. Baseline Model

4.1. Model Description

- **Algorithm:** Weighted Gini impurity (custom implementation).
- **Maximum depth:** None (fully grown).
- **Stopping condition:** Pure node or no improving split, or `max_depth` (pre-pruning via `MaxDepthStoppingCriterion`).
- **Features:** 7.

4.2. Training and Testing Procedure

Same pipeline for all experiments: load, duplicate-group stratified split (75/25, seed 42), train, evaluate. Keeping split identical isolates model effects.

- Training: 531 rows (375 distinct) – Fail 266, Pass 265
- Testing: 177 rows (125 distinct) – Fail 88, Pass 89

Reproduction:

```
python -m src.pure
python -m src.depth
python -m src.depth_cri
```

4.3. Resulting Tree

The baseline tree has 171 nodes, 93 leaves, depth 10. Root split is `Past_Exam_Scores <= 72.5`. The figure shows an excerpt captured from the visualization; the full tree is in the appendix and interactive HTML.

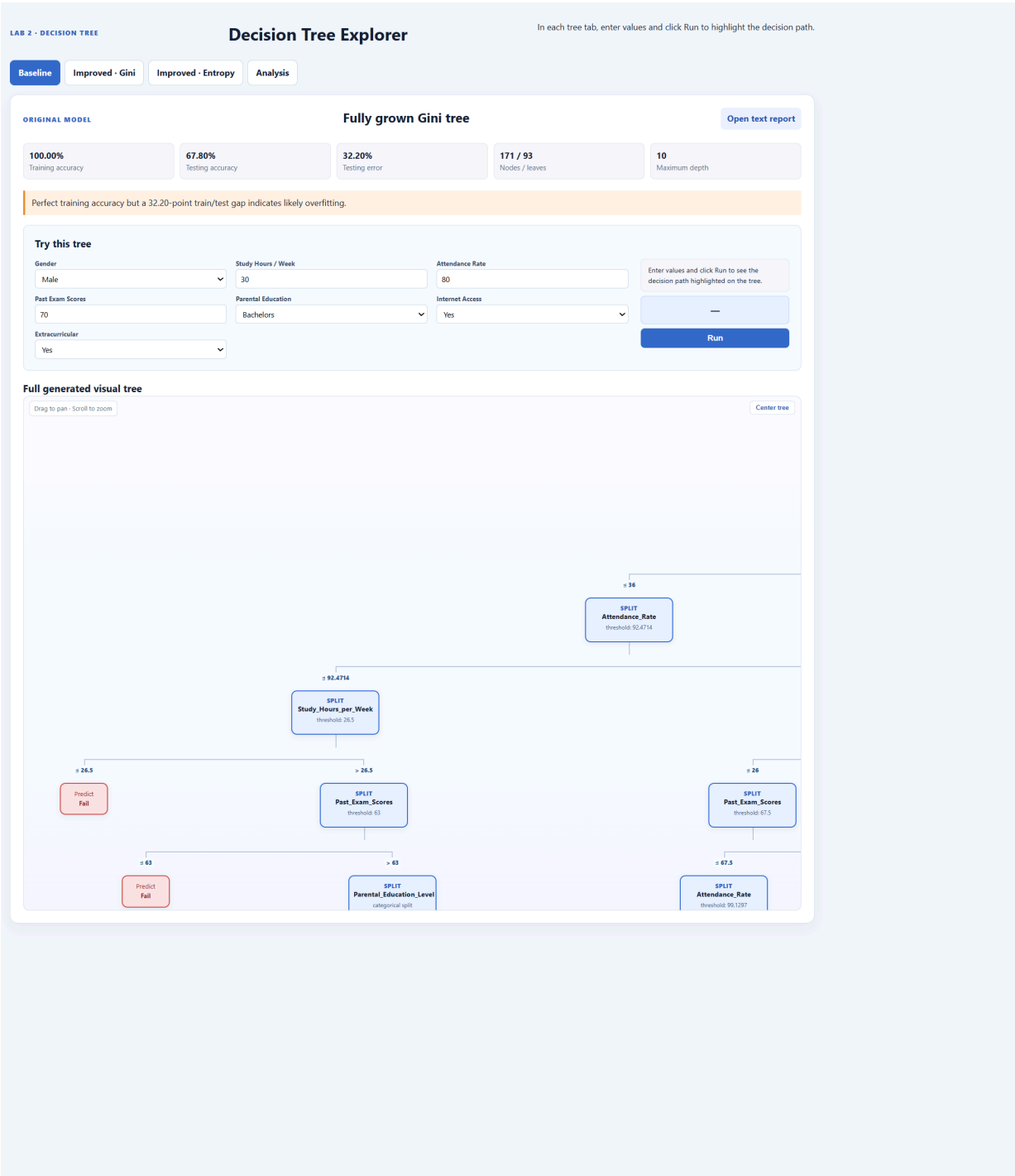


Figure 3. Excerpt of the baseline fully grown tree (171 nodes, depth 10). Full tree in appendix and interactive HTML.

4.4. Accuracy and Error Rate

Dataset	Correct	Incorrect	Accuracy	Error Rate
Training	531/531	0/531	100.00%	0.00%
Testing	120/177	57/177	67.80%	32.20%

Table 6. Baseline accuracy and error rate.

Confusion matrices, with **Actual \ Predicted** layout:

Actual Predicted	Fail	Pass
Fail	266	0
Pass	0	265

Table 7. Training – 100% accuracy.

Actual Predicted	Fail	Pass
Fail	79	9
Pass	48	41

Table 8. Testing – 67.80% accuracy. Fail recall 89.77%, Pass recall 46.07%, gap 32.20 pp.

Extended metrics: Fail Precision 62.20% Recall 89.77% F1 73.49%; Pass Precision 82.00% Recall 46.07% F1 58.99%; Macro F1 66.24%, Balanced accuracy 67.92%. ROC-AUC not reported (hard labels only).

Confusion Matrices on Test Set (Fail 88, Pass 89)

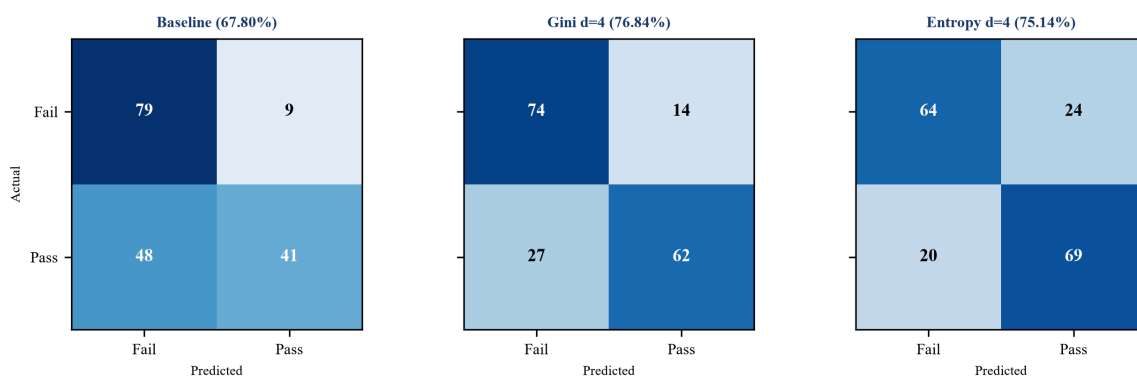


Figure 4. Confusion matrices for the three models on the held-out test set.

5. Analysis of the Resulting Tree

5.1. Overall Structure

The tree is constructed top-down greedily: each node evaluates all value and threshold splits and picks the one most reducing impurity. With depth 10 and 93 leaves, the baseline is much larger than depth-limited models.

5.2. Important Decision Rules

1. **Strongest root-level separation.** Threshold 72.5 selected in all three experiments, indicating best impurity reduction at root.
2. **Study time is conditional.** 36 hours in low-score branch vs 19.5 hours in high-score branch – weaker past scores need more hours to compensate.
3. **Attendance resolves borderline.** Higher attendance generally leads to Pass, but threshold varies by branch.

4. **Deep branches use secondary features.** Gender and other categorical splits at depth 5–10 separate only a few samples (baseline avg 5.7 samples/leaf vs 27.9/31.2 after pruning) and likely capture noise.

5.3. Feature Usage and Impurity Analysis

The baseline tree uses all 7 features but with highly skewed frequency: Attendance_Rate 27 times (34.6% of splits), Study_Hours 19 times (24.4%), Past_Exam_Scores 14 times (17.9%), Gender 9 times (11.5%), Parental_Education 7 times (9.0%), Internet and Extracurricular only once each (1.3%). This confirms numerical features dominate early splits while categorical features are relegated to deep, small leaves. At the root, the Gini impurity drops from 0.5000 (balanced) to a weighted 0.4223 after the Past_Exam_Scores ≤ 72.5 split (reduction 0.0777; Entropy reduction 0.1161, from 1.0000 to 0.8839; left branch $n=202$ Fail 152/Pass 50, right branch $n=329$ Fail 114/Pass 215) – a clear separation that explains why this threshold is consistently chosen.

All 93 leaves of the baseline are pure (100% contain only one class), averaging 5.7 samples/leaf – direct evidence of memorization rather than generalization.

5.4. Strengths and Weaknesses

Strengths: Fully transparent, perfect training fit, intuitive root, captures conditional interactions. **Weaknesses:** Too deep for 7 features, gap 32.20 pp indicates overfitting, biased toward Fail, deep secondary splits likely noise. The tree overfits and motivates pre-pruning.

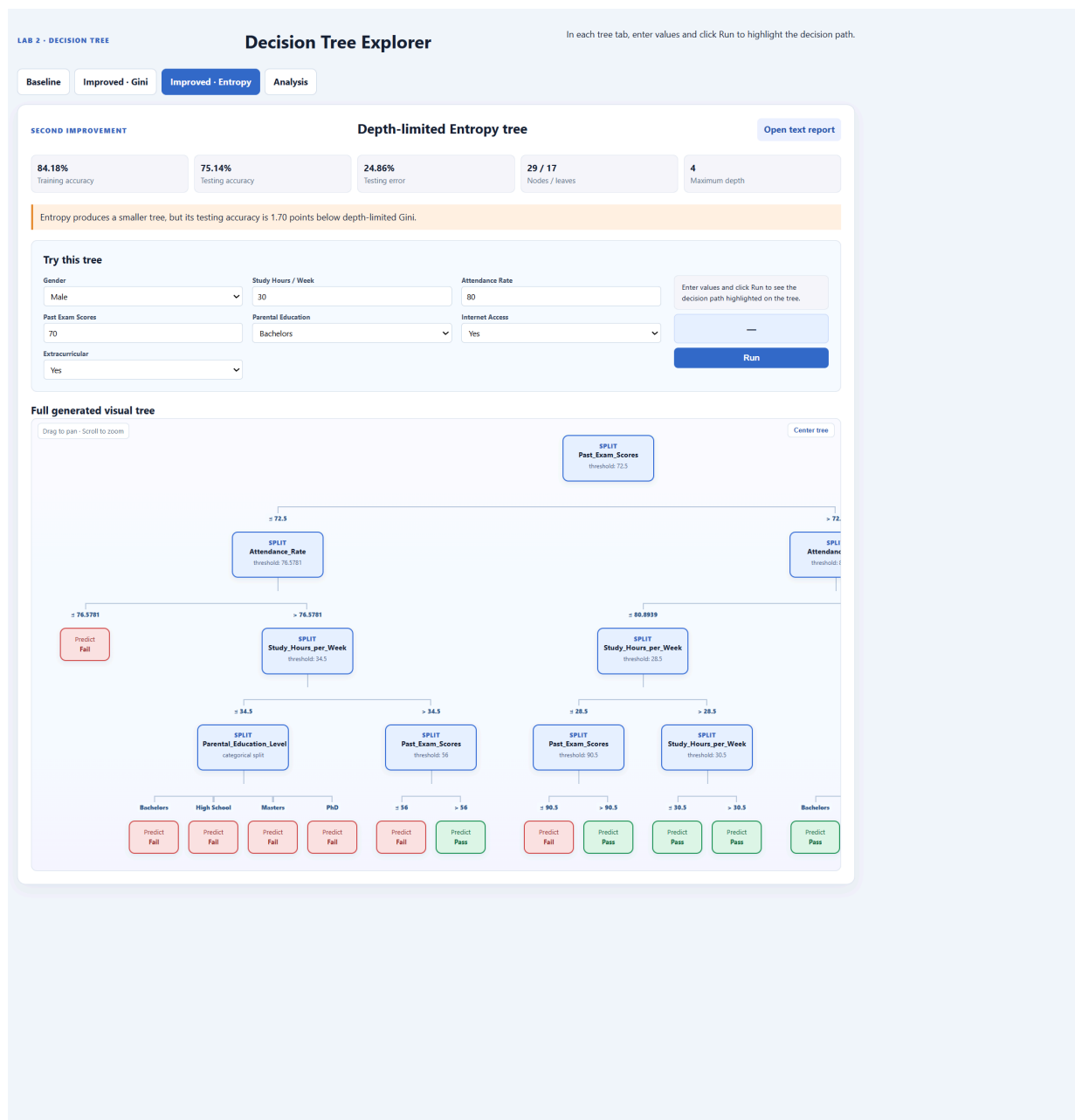


Figure 5. Analysis view from the interactive visualization.

6. Improvement Methods

Two improvements keep data, split, and seed unchanged.

6.1. Improvement 1 – Depth Limiting (Gini, max_depth = 4) – Pre-pruning

Method: Pre-pruned by limiting maximum depth to 4 via `MaxDepthStoppingCriterion`. Other settings remain Gini.

Results: 35 nodes, 19 leaves, depth 4 (approx. 79.5% fewer nodes than baseline). Training 454/531 (85.50% / 14.50%), Testing 136/177 (76.84% / 23.16%).

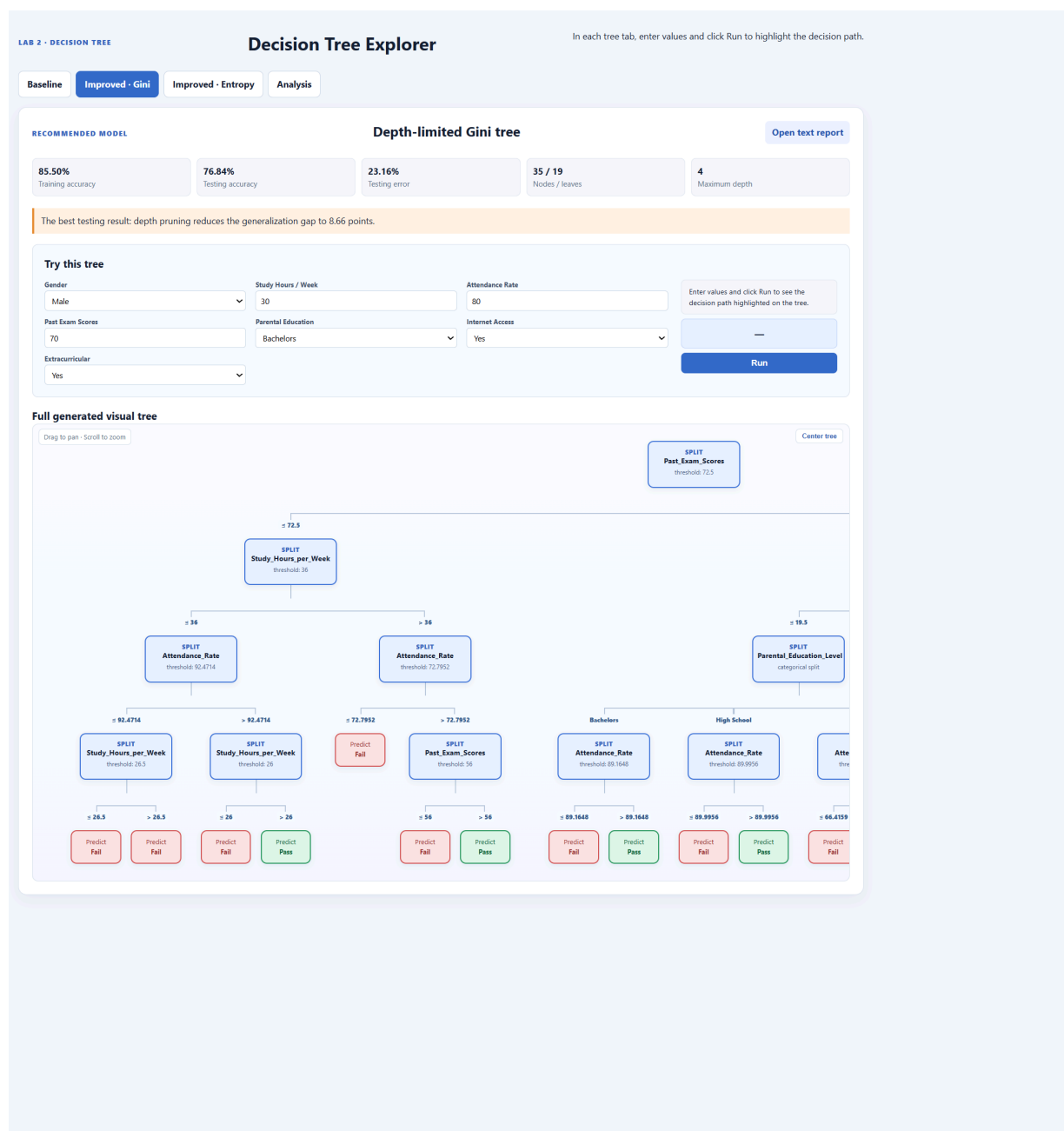


Figure 6. Excerpt of the depth-limited Gini tree (35 nodes, depth 4).

Confusion matrix (held-out test):

Actual Predicted	Fail	Pass
Fail	74	14
Pass	27	62

Table 9. Gini depth=4 – held-out test.

Extended: Fail 73.27/84.09/78.31; Pass 81.58/69.66/75.15; Macro F1 76.73%, Balanced 76.88%. Gap reduced 32.20 -> 8.66 pp, +9.04 pp test. Bias-variance trade-off: larger leaves (27.9 vs 5.7 samples/leaf) reduce variance.

6.2. Improvement 2 – Criterion Change (Gini to Entropy, depth = 4)

Method: Keep depth 4, change Gini to Entropy.

Results: 29 nodes, 17 leaves, depth 4 – smallest. Training 447/531 (84.18% / 15.82%), Testing 133/177 (75.14% / 24.86%).

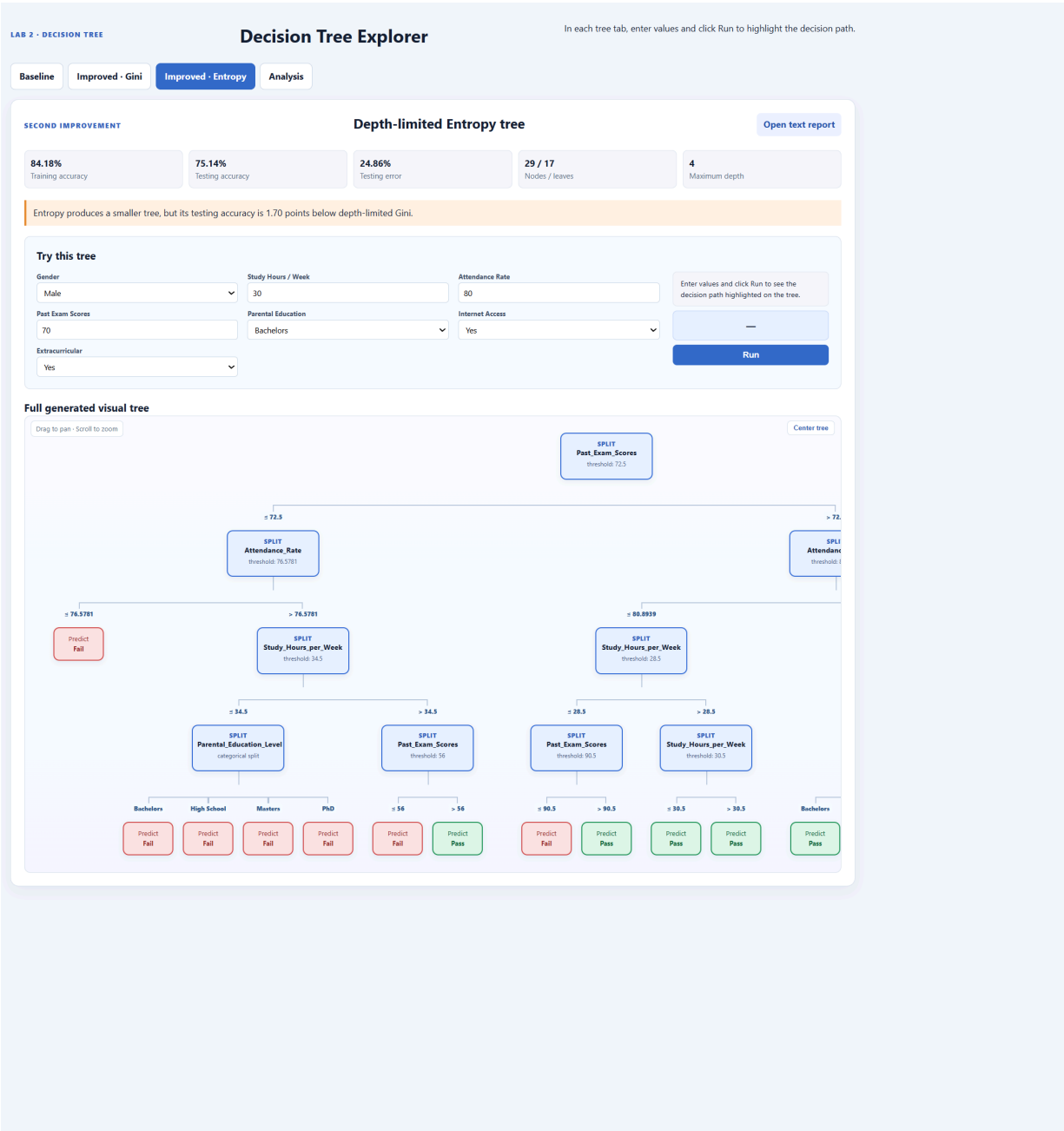


Figure 7. Excerpt of the depth-limited Entropy tree (29 nodes, depth 4).

Confusion matrix (held-out test):

Actual Predicted	Fail	Pass
	64	24
Pass	20	69

Table 10. Entropy depth=4 – held-out test.

Extended: Fail 76.19/72.73/74.42; Pass 74.19/77.53/75.82; Macro F1 75.12%.

Criterion (depth 4)	Fail recall	Pass recall	Test Accuracy
Gini	74/88 (84.09%)	62/89 (69.66%)	76.84%
Entropy	64/88 (72.73%)	69/89 (77.53%)	75.14%

Table 11. Gini vs Entropy at depth 4.

Entropy is slightly less effective overall (-1.70 pp, 3 more errors) on this balanced data, is more sensitive to pure splits, creates smaller tree (29 vs 35 nodes) and earlier Attendance splits, helping Pass recall (+7) but harming Fail recall (-10).

7. Comparison of Results

Model	Criterion	Depth	Nodes	Leaves	Depth	Train Acc.	Train Err.	Test Acc.	Test Err.	Gap
Baseline	Gini	None	171	93	10	100.00%	0.00%	67.80%	32.20%	32.20
Depth-limited	Gini	4	35	19	4	85.50%	14.50%	76.84%	23.16%	8.66
Depth-limited	Entropy	4	29	17	4	84.18%	15.82%	75.14%	24.86%	9.04

Table 12. Overall comparison on the held-out test set. Best in bold. Nodes decrease by approx. 79.5% (171 - > 35).

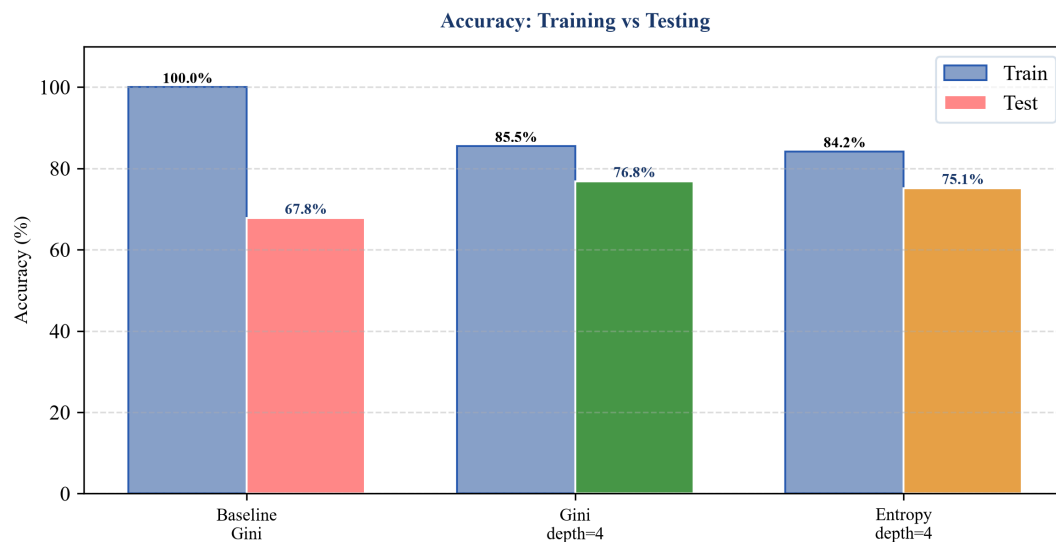


Figure 8. Training vs held-out testing accuracy and gap.

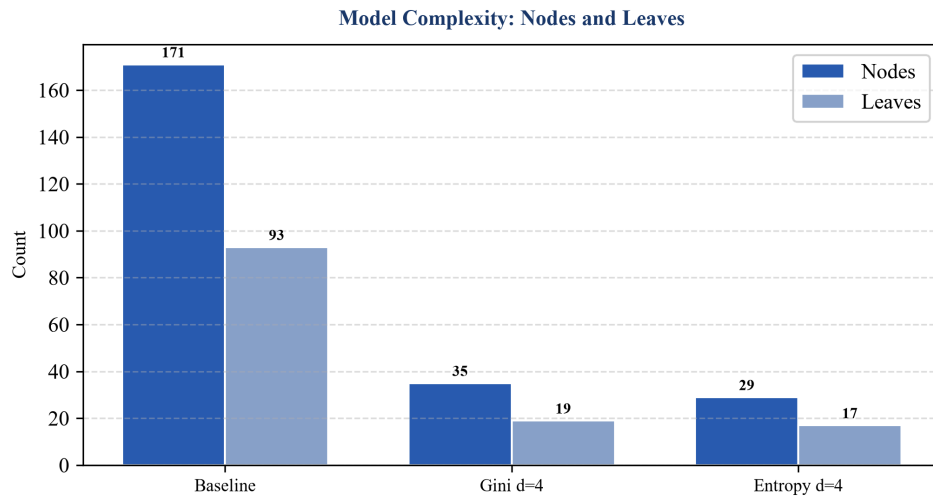


Figure 9. Model complexity – pre-pruning reduces size by about 80%.

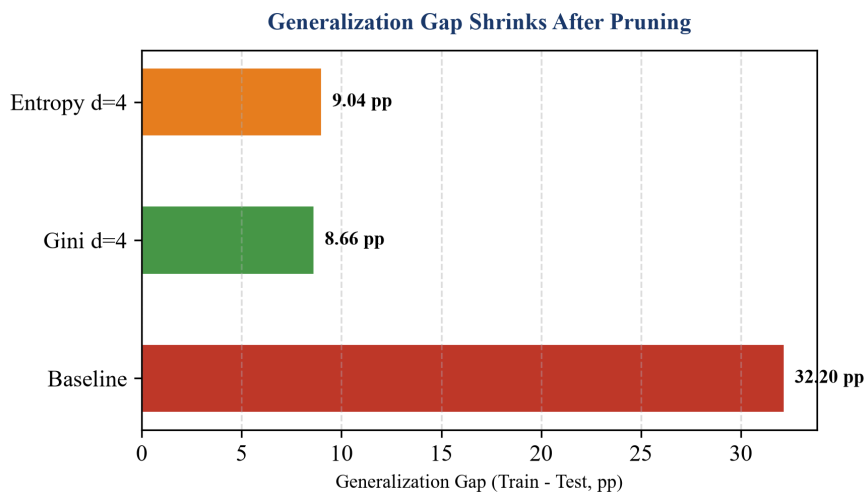


Figure 10. Generalization gap reduced from 32.20 to 8.66 points.

Best on held-out test: depth-limited Gini, max_depth=4 (76.84%, 23.16% error).

8. Conclusion

This project shows how decision trees learn thresholds directly from data and how easily a fully grown tree can overfit (100% train vs 67.80% test, 171 nodes). Limiting depth to 4 with pre-pruning proved most effective: removed 136 nodes (approx. 79.5% fewer), reduced gap 32.20 -> 8.66, raised held-out test to 76.84%. Changing Gini to Entropy at same depth produced a slightly smaller tree with different recall trade-off but did not beat Gini. Recommended: Gini, max_depth=4 (76.84% on held-out test, 23.16% error, 35 nodes, 19 leaves). With depth limiting, trees reach about 77% on this held-out split, suggesting usefulness for early warning while remaining transparent. This is limited to the reported split.

9. References

1. Lab 2 – Decision Tree Modeling and Improvement, HCMUS.
2. Dataset: Amr Maree – Student Performance Prediction, Kaggle. <https://www.kaggle.com/datasets/amrmaree/student-performance-prediction> (local copy: data/student_data.csv, 708 rows, 500 distinct).
3. Breiman et al. – Classification and Regression Trees (1984).
4. Quinlan – Induction of Decision Trees (1986) and Mitchell – Machine Learning, Ch. 3 (1997).
5. Scikit-learn – Decision Trees (background).
6. Project implementation: Custom framework in model/ and src/ utils.py, src/ pure.py, src/ depth.py, src/ depth_cri.py.
7. Visualization: docs/tree_visualization.html and report/assets/ (1.png-4.png and generated charts).

10. Appendices

10.1. Appendix A – Baseline Tree (detailed excerpt)

The full baseline tree has 171 nodes, 93 leaves, depth 10. Top 4 levels:

```
Split on Past_Exam_Scores at 72.5
├── If <= 72.5: Split on Study_Hours_per_Week at 36
│   ├── If <= 36: Split on Attendance_Rate at 92.47
│   │   ├── If <= 92.47: Split on Study_Hours at 26.5 ...
│   │   └── If > 36: Split on Attendance_Rate at 72.79 ...
│   └── If > 72.5: Split on Study_Hours at 19.5 ...
└── Leaf avg 5.7/leaf (4.0 distinct) - many 1-2 sample leaves.
```

Full text: docs/RESULT_PURE.md. Interactive: docs/tree_visualization.html and report/assets/1.png.

10.2. Appendix B – Depth-limited Gini Tree

Top 4 levels (35 nodes, 19 leaves):

```
Split on Past_Exam_Scores at 72.5
├── If <= 72.5: Split on Study_Hours at 36 ...
└── If > 72.5: Split on Study_Hours at 19.5 ...
    Leaf avg 27.9/leaf (19.7 distinct).
```

Full text: docs/RESULT_DEPTH.md and report/assets/2.png.

10.3. Appendix C – Depth-limited Entropy Tree

Top 4 levels (29 nodes, 17 leaves):

```
Split on Past_Exam_Scores at 72.5
├── If <= 72.5: Split on Attendance at 76.57 ...
└── If > 72.5: Split on Attendance at 80.89 ...
    Leaf avg 31.2/leaf (22.0 distinct).
```

Full text: docs/RESULT_DEPTH_CRI.md and report/assets/3.png.

10.4. Appendix D – Reproduction

From repository root (verified):

```
python -m src.pure
python -m src.depth
python -m src.depth_cri
Outputs: docs/RESULT_PURE.md, docs/RESULT_DEPTH.md, docs/RESULT_DEPTH_CRI.md
Assets: python report/assets/generate_assets.py
Report: typst compile report/Report.typ report/Report.pdf
```

Settings: duplicate-group aware split, 75/25, seed 42, Gini/Entropy, max_depth, AttributeValueSplit/ThresholdSplit, pre-pruning only.